

Homework Assignment: Solution Set

Advanced Data Structures and Algorithms

Department of Computer Science

Question 1

Problem: Prove that the time complexity of the recursive HEAPIFY operation is $O(\log n)$ using the recurrence relation:

$$T(n) \leq T\left(\frac{2n}{3}\right) + O(1).$$

Detailed Solution:

We are analyzing the recurrence $T(n) = T(2n/3) + c$, where c is a constant representing the comparison and swapping operations at the current node.

Why $2n/3$? In the worst-case scenario for a binary heap (which is nearly complete), the left subtree can be larger than the right subtree. Specifically, the worst case occurs when the bottom level is exactly half full. In this scenario, the left subtree contains roughly $2n/3$ nodes, while the right contains $1n/3$.

Step 1: Unrolling the Recurrence We solve this by the substitution method (or unrolling). At each step, we descend into the larger subtree, doing constant work c :

$$\begin{aligned} T(n) &= T\left(\frac{2}{3}n\right) + c \\ &= \left[T\left(\frac{2}{3} \cdot \frac{2}{3}n\right) + c\right] + c = T\left(\left(\frac{2}{3}\right)^2 n\right) + 2c \\ &= T\left(\left(\frac{2}{3}\right)^3 n\right) + 3c \\ &\vdots \\ &= T\left(\left(\frac{2}{3}\right)^k n\right) + k \cdot c \end{aligned}$$

Step 2: Finding the Termination Condition The recursion stops when the problem size reduces to the base case, i.e., $n_{final} \approx 1$.

$$\left(\frac{2}{3}\right)^k n = 1 \implies n = \left(\frac{3}{2}\right)^k$$

Taking the logarithm (base $3/2$) on both sides to solve for the depth k :

$$k = \log_{3/2} n$$

Using the change-of-base formula, we know that $\log_{3/2} n = \frac{\ln n}{\ln 1.5}$. Since $\ln 1.5$ is a constant, k grows proportionally to $\ln n$ (or $\log_2 n$). Thus, $k = \Theta(\log n)$.

Step 3: Final Complexity Substituting k back into our unrolled equation:

$$T(n) = T(1) + c \cdot \Theta(\log n) = O(\log n)$$

This proves that even in the worst-case structural imbalance of a binary heap, the Heapify operation remains logarithmic.

Question 2

Problem: In an array of size n representing a binary heap, prove that all leaf nodes are located at indices from $\lfloor n/2 \rfloor + 1$ to n .

Detailed Solution:

In a standard binary heap implementation using 1-based indexing:

- For a node at index i , the **Left Child** is at index $2i$.
- For a node at index i , the **Right Child** is at index $2i + 1$.

Definition of a Leaf: A node is a leaf if and only if it has no children. This means the index of its potential left child must be outside the bounds of the array.

$$\text{Node } i \text{ is a leaf} \iff \text{LeftChild}(i) > n$$

Substituting the formula for the left child:

$$2i > n \implies i > \frac{n}{2}$$

Since array indices are integers, the smallest integer strictly greater than $n/2$ is $\lfloor \frac{n}{2} \rfloor + 1$.

Conclusion: Therefore, the first node that is a leaf is at index $\lfloor n/2 \rfloor + 1$. All subsequent indices up to n ($i > n/2$) satisfy the condition $2i > n$. Thus, the leaves are located at:

$$\lfloor n/2 \rfloor + 1, \dots, n.$$

Question 3

Problem:

- (a) Show that in any heap containing n elements, the number of nodes at height h is at most $\lceil n/2^{h+1} \rceil$.
- (b) Using this, prove that BUILD-HEAP is $O(n)$.

Detailed Solution:

(a) **Nodes at height h** Let's define the height of a leaf as 0.

- At height $h = 0$ (leaves), we just proved (in Q2) that there are $\lceil n/2 \rceil$ nodes (roughly half).
- At height $h = 1$, we are looking at the parents of the leaves. There are roughly $n/4$ such nodes.

By induction, every time we move up a level in a binary tree, the number of nodes reduces by a factor of 2. Therefore, the number of nodes at height h is bounded by:

$$\text{Count}(h) \leq \frac{n}{2^{h+1}}$$

(b) **Complexity of Build-Heap** The BUILD-HEAP algorithm iterates from the last non-leaf node down to 1, calling HEAPIFY on each.

- The cost of HEAPIFY on a node at height h is $O(h)$ (it bubbles down at most h levels).
- We sum the cost over all nodes, grouping them by height.

The total work $T(n)$ is the sum over all possible heights (from 0 to $\lfloor \log n \rfloor$):

$$T(n) = \sum_{h=0}^{\lfloor \log n \rfloor} (\text{number of nodes at } h) \cdot O(h)$$

Substituting our bound from part (a):

$$T(n) \leq \sum_{h=0}^{\lfloor \log n \rfloor} \frac{n}{2^{h+1}} \cdot h = \frac{n}{2} \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^h}$$

We treat $n/2$ as a constant relative to the summation. We focus on the series $\sum \frac{h}{2^h}$. This is an Arithmetico-Geometric Series that converges to a constant (specifically, $\sum_{h=0}^{\infty} \frac{h}{2^h} = 2$).

$$T(n) \leq \frac{n}{2} \cdot O(1) = O(n)$$

Thus, building a heap is a linear time operation, $O(n)$, which is more efficient than calling insert n times ($O(n \log n)$).

Question 4

Problem: Explain the LU decomposition of a matrix using Gaussian Elimination.

Detailed Solution:

LU decomposition is the process of factoring a square matrix A into two triangular matrices:

$$A = L \cdot U$$

where L is **Lower Triangular** (with 1s on the diagonal) and U is **Upper Triangular**.

Connection to Gaussian Elimination: Gaussian Elimination transforms A into row-echelon form (which is exactly U) by subtracting multiples of rows from others to create zeros below the diagonal.

1. **The Multipliers:** When we eliminate $A[i, k]$ using the pivot $A[k, k]$, we calculate a multiplier:

$$m_{ik} = \frac{A[i, k]}{A[k, k]}$$

2. **The Operation:** We update row i : $Row_i \leftarrow Row_i - m_{ik} \cdot Row_k$.
3. **Storing History:** The "magic" of LU decomposition is that these multipliers m_{ik} are exactly the values needed for the matrix L . Specifically, $L[i, k] = m_{ik}$.

Basically, U is the result of elimination, and L is the record of the transformations (multipliers) used to get there.

Question 5

Problem: Solve the recurrence relation from LUP decomposition:

$$T(n) = \sum_{i=1}^n \left[O(1) + \sum_{j=1}^{i-1} O(1) \right] + \sum_{i=1}^n \left[O(1) + \sum_{j=i+1}^n O(1) \right]$$

Detailed Solution:

This recurrence represents the computational cost of **Forward Substitution** and **Backward Substitution** (solving $Ly = b$ and $Ux = y$) rather than the $O(n^3)$ decomposition itself. Let's evaluate the sums.

Part 1: Forward Substitution ($Ly = b$) The first term is:

$$\sum_{i=1}^n \left[1 + \sum_{j=1}^{i-1} 1 \right]$$

The inner sum runs $i - 1$ times (constant work per element in the row).

$$= \sum_{i=1}^n (1 + i - 1) = \sum_{i=1}^n i$$

This is the sum of the first n integers:

$$= \frac{n(n+1)}{2} = \Theta(n^2)$$

Part 2: Backward Substitution ($Ux = y$) The second term is:

$$\sum_{i=1}^n \left[1 + \sum_{j=i+1}^n 1 \right]$$

The inner sum runs from $i + 1$ to n , which is $(n - (i + 1) + 1) = n - i$ times.

$$= \sum_{i=1}^n (1 + n - i)$$

Let $k = n - i$. As i goes $1 \rightarrow n$, k goes $n - 1 \rightarrow 0$.

$$= \sum_{k=0}^{n-1} (1 + k) \approx \frac{n^2}{2} = \Theta(n^2)$$

Total Complexity:

$$T(n) = \Theta(n^2) + \Theta(n^2) = \boxed{O(n^2)}$$

Question 6

Problem: Prove that if matrix A is non-singular, then its Schur complement is also non-singular.

Detailed Solution:

Let A be partitioned into block matrices:

$$A = \begin{bmatrix} B & C \\ D & E \end{bmatrix}$$

where B is a square, invertible submatrix. The **Schur Complement** of block B is defined as:

$$S = E - DB^{-1}C$$

We use the known determinant identity for block matrices. A can be factored into block-triangular forms, leading to the relation:

$$\det(A) = \det(B) \cdot \det(E - DB^{-1}C) = \det(B) \cdot \det(S)$$

Proof Logic:

1. We are given that A is **non-singular**, which means $\det(A) \neq 0$.
 2. From the equation $\det(A) = \det(B) \cdot \det(S)$, for the product to be non-zero, **both** factors must be non-zero.
 3. Therefore, $\det(S) \neq 0$.
 4. A matrix with a non-zero determinant is non-singular. Thus, the Schur complement S is non-singular.
-

Question 7

Problem: Prove that positive-definite matrices are suitable for LU decomposition and do not require pivoting.

Detailed Solution:

What is the risk without pivoting? In standard Gaussian elimination, we need pivoting (swapping rows) if a pivot element a_{kk} is 0 (to avoid division by zero) or very small (to avoid numerical instability).

Properties of Positive Definite (PD) Matrices: A symmetric matrix A is Positive Definite if $x^T Ax > 0$ for all non-zero vectors x . A key property of PD matrices is that **every leading principal submatrix is also positive definite**. This implies that the determinant of every leading principal submatrix is strictly positive.

Why this implies no pivoting needed: During the k -th step of Gaussian elimination, the pivot element u_{kk} is determined by the ratio of determinants of the leading submatrices:

$$u_{kk} = \frac{\det(A_k)}{\det(A_{k-1})}$$

Since A is PD, both $\det(A_k) > 0$ and $\det(A_{k-1}) > 0$. Consequently, the pivot u_{kk} is always strictly positive (and specifically, never zero). Since the pivot is never zero, the algorithm proceeds safely without needing to swap rows.

Question 8

Problem: For finding an augmenting path in a graph, should BFS or DFS be applied? Justify.

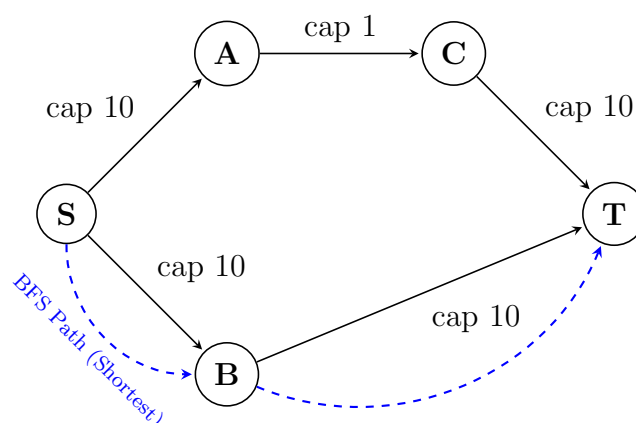
Detailed Solution:

Answer: Breadth First Search (BFS) is the correct choice. When used in the Ford-Fulkerson method, this implementation is known as the **Edmonds-Karp algorithm**.

Justification:

1. **Shortest Path Guarantee:** BFS explores the graph layer by layer. It is guaranteed to find the augmenting path with the **minimum number of edges**.
2. **Termination bound:** It can be proven that if we always choose the shortest path, the total number of augmentations is bounded by $O(VE)$. This ensures the algorithm terminates efficiently.
3. **Problems with DFS:** Depth First Search might choose a very long, winding path. In cases with irrational capacities, DFS can actually fail to terminate. Even with integer capacities, DFS might choose paths that only increase flow by 1 unit at a time, leading to exponential runtime (e.g., the classic pathological example with a large capacity bridge).

Visual Comparison: In the graph below, BFS finds the direct path $S \rightarrow B \rightarrow T$ (2 edges). DFS might meander $S \rightarrow A \rightarrow C \rightarrow T$, utilizing edges that might be bottlenecks in a less optimal way.



Question 9

Problem: Explain why Dijkstra's algorithm cannot be applied to graphs with negative edge weights.

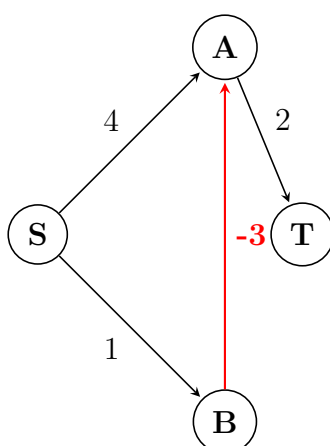
Detailed Solution:

Dijkstra's algorithm relies on the **Greedy Choice Property**. It assumes that once a node is "settled" (extracted from the priority queue), its shortest distance is finalized and will never decrease.

The Failure Mode: If edges are non-negative, extending a path always increases (or keeps constant) the total cost. However, a negative edge can reduce the total cost. If Dijkstra settles node u , and later discovers a path to a neighbor v that leads *back* to u via a negative edge with a lower total cost, the algorithm fails because it never re-evaluates settled nodes.

Example: Consider $S \rightarrow A$ (cost 5), $S \rightarrow B$ (cost 2), and $B \rightarrow A$ (cost -10).

1. Dijkstra picks B first (cost 2) and settles it.
2. It picks A next (cost 5) via the direct edge and settles it.
3. It *then* explores B 's edges, finding $B \rightarrow A$ with weight -10. The path $S \rightarrow B \rightarrow A$ costs $2 + (-10) = -8$.
4. But A is already "settled" at cost 5. Dijkstra will not update it. The result is incorrect.



Question 10

Problem: Prove that every connected component of the symmetric difference of two matchings is a path or an even cycle.

Detailed Solution:

Let M_1 and M_2 be two matchings in a graph G . Consider the graph G' formed by the edges in their symmetric difference $M_1 \oplus M_2$ (edges in M_1 or M_2 but not both).

Degree Analysis:

- In any matching, a vertex is incident to at most 1 edge.
- In G' , a vertex can be incident to at most one edge from M_1 and at most one edge from M_2 .
- Therefore, the degree of any vertex in G' is either 0, 1, or 2.

Structure of Components: A connected graph where every vertex has a degree ≤ 2 must be either:

1. **An isolated vertex** (degree 0).
2. **A path** (endpoints have degree 1, internal nodes degree 2).
3. **A cycle** (all nodes have degree 2).

Why Cycles must be Even: In a path or cycle in G' , the edges must alternate between belonging to M_1 and M_2 (since two M_1 edges cannot touch at a vertex). For a cycle to close back on itself, it must start with an edge from one set (say M_1) and end with an edge from the other (M_2). This requires pairs of edges, implying the total length must be even.

Question 11

Problem: Define Co-NP.

Detailed Solution:

Definition: Co-NP is the complexity class of decision problems where the **complement** of the problem is in NP.

Intuition - Verification:

- For **NP** problems, if the answer is "YES", there exists a certificate (proof) that can be verified in polynomial time.
- For **Co-NP** problems, if the answer is "NO", there exists a **counter-example** that can be verified in polynomial time.

Example: The problem TAUTOLOGY: "Is this boolean formula true for *all* assignments?" This is in Co-NP. Why? If the answer is "NO", I can simply show you **one** assignment of variables that makes the formula false. You can verify this counter-example efficiently.

Question 12

Problem: Verify Boolean circuit output using DFS.

Detailed Solution:

A Boolean circuit is essentially a **Directed Acyclic Graph (DAG)** where nodes are logic gates (AND, OR, NOT) and inputs. We want to find the value of the output node.

Algorithm: We can use a Depth First Search approach with **Memoization**:

1. Start DFS at the output gate.
2. For the current gate, recursively call DFS on its input wires (children in the DAG).
3. Once the values of the children are returned, compute the logic operation (e.g., if AND gate, return $val(child1) \wedge val(child2)$).
4. **Crucial Step:** Mark the node as "visited" and store its computed value. If we encounter this node again (via a different wire), return the stored value immediately.

Complexity: Because it is a DAG and we use memoization, every edge and node is processed exactly once. The time complexity is linear with respect to the size of the circuit: $O(V + E)$.

Question 13

Problem: Is 3-SAT NP-Hard?

Detailed Solution:

Yes, 3-SAT is NP-Hard.

Reasoning:

1. We know by the **Cook-Levin Theorem** that general Boolean Satisfiability (SAT) is NP-Complete (and thus NP-Hard).
 2. We can reduce generic SAT to 3-SAT in polynomial time ($\text{SAT} \leq_p \text{3-SAT}$).
 3. The reduction involves taking any long clause $(x_1 \vee x_2 \vee \dots \vee x_k)$ and breaking it down into a chain of clauses of size 3 using new dummy variables.
 4. Since a known NP-Hard problem can be reduced to 3-SAT, 3-SAT must be at least as hard. Therefore, it is NP-Hard.
-

Question 14

Problem: Is 2-SAT NP-Hard?

Detailed Solution:

No, 2-SAT is not NP-Hard. It is actually solvable in polynomial time (specifically, linear time $O(V + E)$).

How to solve 2-SAT efficiently: We can model the problem using an **Implication Graph**.

- A clause $(A \vee B)$ is logically equivalent to $(\neg A \implies B)$ and $(\neg B \implies A)$.
- We build a directed graph where nodes are literals $(x, \neg x)$.
- The formula is unsatisfiable if and only if there is a path from x to $\neg x$ **and** a path from $\neg x$ to x .

In graph theory terms, this means x and $\neg x$ belong to the same **Strongly Connected Component (SCC)**. We can find SCCs using Tarjan's algorithm or Kosaraju's algorithm in linear time. Thus, 2-SAT is in **P**.